

EXPERIMENT-2

StudyHub — Student Task Manager (Single-Page App)

Scenario

You are building StudyHub, a small productivity app that helps students manage their daily tasks. This version is a **single page** — there is no routing and no separate Login/Home/Tasks/Profile pages. Everything lives on one screen, and your job is to focus on **state management and performance patterns**: Context API with a custom hook, `useReducer`, `useMemo`, and `React.memo`.

Given Data

Assume the student's information is:

None

```
const student = {  
  name: "Priya Nair",  
  email: "priya@gmail.com",  
  year: "3rd Year",  
};
```

Assume the student starts with these tasks:

None

```
const initialTasks = [  
  { id: 1, title: "Finish DBMS assignment", completed: false },  
  { id: 2, title: "Revise React hooks", completed: false },  
  { id: 3, title: "Submit lab report", completed: true },  
];
```

Component Structure

You may follow this structure, or adapt it as long as the same responsibilities are kept separate:

```
None
App
|
|-- StudentProvider
|
|   |-- Header    (app title + shows student name via useUser())
|
|   |-- ProfilePanel      (uses useUser() custom hook)
|
|   |-- TaskManager      (uses useReducer)
|
|       |-- TaskStats    (uses useMemo, derived from tasks)
|       |-- AddTaskForm
|       |-- TaskList
|
|           |-- TaskItem  (wrapped in React.memo)
```

There is no `<BrowserRouter>`, no `<Routes>`, no `<Navigate>`, and no `useNavigate()` in this version — everything renders on one page at all times.

Requirements

1. Context API + Custom Hook for Student Info

- Create a `StudentContext` that stores the student object (name, email, year).
- Provide it once, near the top of the component tree (e.g. wrap the whole app in a `StudentProvider`).
- Create a custom hook called `useUser()` that calls `useContext(StudentContext)` internally.

- Any component that needs student info (e.g. Header, ProfilePanel) must call `useUser()` directly — student data should **not** be passed down as props.

2. Task Management with `useReducer`

- Inside `TaskManager`, manage the task list with a single `useReducer` instead of separate `useState` calls.
- Your reducer must support at least these action types:
 - `SET_TASKS` — loads the initial tasks into state.
 - `ADD_TASK` — adds a new task from a text input.
 - `TOGGLE_TASK` — flips a task's completed status by id.
 - `DELETE_TASK` — removes a task by id.
- Example dispatch call:

None

```
dispatch({
  type: "TOGGLE_TASK",
  payload: taskId,
});
```

- `TaskManager` should render an `AddTaskForm` (text input + "Add Task" button) and a `TaskList`.

3. Derived Data with `useMemo`

- Add a `TaskStats` display (e.g. "2 remaining / 3 total") that is computed from the tasks array using `useMemo`, so the count is only recalculated when the tasks array actually changes — not on every render (e.g. not on every keystroke in the "Add Task" input).
- Optionally, also use `useMemo` to compute a filtered/sorted list of tasks (e.g. incomplete tasks first) so the derived array isn't rebuilt unnecessarily on unrelated re-renders.

4. Preventing Unnecessary Re-renders with `React.memo`

- Wrap `TaskItem` in `React.memo` so that typing in the `AddTaskForm` input (which causes `TaskManager` to re-render) does **not** re-render every `TaskItem` — only the item whose own props actually changed should re-render.
- Pass stable callback references (e.g. via `useCallback`, or by passing `dispatch` down directly) so `React.memo` is actually effective and props don't appear "new" on every render.

5. Custom Hooks

- Build at least **two** custom hooks:
 1. `useUser()` — reads the student info from `StudentContext` (see Requirement 1).
 2. `useTaskStats(tasks)` — takes the tasks array and returns derived values (e.g. `{ total, completed, remaining }`) using `useMemo` internally. `TaskStats` should call this hook rather than computing the numbers inline.

Expected Behaviour

On load, the student should see everything on a single page:

None

STUDYHUB

Welcome, Priya Nair (3rd Year)

Student Details

Name: Priya Nair

Email: priya@gmail.com

Year: 3rd Year

MY TASKS (2 remaining / 3 total)

New task: _____ [Add Task]

[] Finish DBMS assignment [Delete]

[] Revise React hooks

[Delete]

[x] Submit lab report

[Delete]

Typing in the "New task" input should update the input's own state without causing every `TaskItem` in the list to re-render — this is the behavior `React.memo` and `useMemo` in this assignment are meant to demonstrate.

State

https://docs.google.com/presentation/d/1AgDy54_bzFRVu1PtQ_YgSIUqVcFKQvDFSBBPctHrR6g/edit?slide=id.g3b7af23920d_0_38#slide=id.g3b7af23920d_0_38

Why React State is Immutable:

1. What is Immutability?

In programming, **immutable** means "unable to be changed after creation."

- When an object or array is immutable, you cannot alter its properties or elements directly.
- Instead of modifying the existing data structure, you **create a new copy** with the updated values.

2. Why React Demands Immutability

React's core mission is to keep your application's UI in sync with your application's state. To do this efficiently, React must quickly answer one question: **"Did the state change?"**

Reason A: Reference Equality Checks (Shallow Checking)

If React had to check every property of a deeply nested object to see if something changed, it would be extremely slow:

```
// Deep Comparison (Slow: O(N))
if (prevUser.address.geo.lat !== newUser.address.geo.lat) { ... }
```

Instead, React performs a **Shallow Comparison** (using `Object.is` or simple `===` checks on the reference pointer in memory):

```
// Shallow Comparison (Instant: O(1))
if (prevUser !== newUser) {
  // Re-render component!
}
```

- If you mutate an object in-place, the memory address (reference) stays the exact same. React sees `prevState === newState`, assumes nothing changed, and **skips re-rendering**.

- If you copy the object, it gets a **new memory address**. React sees `prevState !== newState` and instantly schedules a UI update.

3. Code Demonstration: Mutable vs. Immutable

Let's look at what happens in memory with two different approaches.

The Wrong Way (Direct Mutation)

```
import { useState } from 'react';

export default function Profile() {
  const [user, setUser] = useState({ name: 'Alex', age: 20 });

  const handleUpdate = () => {
    // WRONG: Mutating the state object directly
    user.age = 21;

    // Telling React to update, but passing the SAME object reference
    setUser(user);
  };

  return <button onClick={handleUpdate}>Age: {user.age}</button>;
}
```

What happens internally (Direct Mutation):

1. The user object is stored at memory address 0x111 (containing { name: 'Alex', age: 20 }).
2. `user.age = 21` changes the age property directly inside address 0x111.
3. `setUser(user)` is called, passing the object at address 0x111.
4. React compares the current state pointer with the incoming pointer:
`Object.is(0x111, 0x111)` // Evaluates to true
5. Since the references are identical, **React bails out (does not trigger a re-render)**. The UI remains stuck showing "Age: 20" even though the variable changed.

The Correct Way (Immutable Update)

```
import { useState } from 'react';

export default function Profile() {
```

```
const [user, setUser] = useState({ name: 'Alex', age: 20 });

const handleUpdate = () => {
  // Correct: Creating a new copy with the spread operator (...)
  const updatedUser = { ...user, age: 21 };

  // Passing the NEW object reference to the setter
  setUser(updatedUser);
};

return <button onClick={handleUpdate}>Age: {user.age}</button>;
}
```

What happens internally (Immutable Update):

1. The initial user object is stored at memory address 0x111 ({ name: 'Alex', age: 20 }).
2. { ...user, age: 21 } allocates a **new object** in memory at a brand-new address, e.g., 0x222 ({ name: 'Alex', age: 21 }).
3. setUser(updatedUser) is called, passing the object at address 0x222.
4. React compares the pointers: Object.is(0x111, 0x222) // Evaluates to false
5. Since they are different, **React schedules a re-render**. The UI updates to show "Age: 21".

4. Visualizing References in Memory

You can write this simple breakdown on a whiteboard for your students:

Step	State Action	Value	Memory Reference Pointer	React Reaction
Start	Initial Render	{ name: 'Alex', age: 20 }	0x111 (Original)	-

A	Direct Mutation	{ name: 'Alex', age: 21 }	0x111 (Unchanged pointer)	No Render (References matched)
B	Immutable Update	{ name: 'Alex', age: 21 }	0x222 (New allocation)	Re-renders UI (Pointers differed)

Hooks

What are Hooks?

Hooks are special functions provided by React that add extra capabilities to functional components, such as state management, lifecycle handling, DOM access, and performance optimization, without using class components.

Why do we use Hooks?

Imagine you want to build a counter.

Without Hooks

None

```
function App() {  
  let count = 0;  
  return (  
    <>  
      <h1>{count}</h1>  
      <button>Increment</button>  
    </>  
  );  
}
```

Clicking the button won't update the UI because count is just a normal JavaScript variable. React doesn't know it changed.

Using useState

None

```
import { useState } from "react";

function App() {
  const [count, setCount] = useState(0);
  return (
    <>
      <h1>{count}</h1>

      <button onClick={() => setCount(count +
1)}>
        Increment
      </button>
    </>
  );
}
```

Now React knows when the state changes and automatically re-renders the component.

The Rules of Hooks

Before diving into individual hooks, you must adhere to the two rules that govern them:

1. **Only Call Hooks at the Top Level:** Don't call Hooks inside loops, conditions, or nested functions. This ensures Hooks are called in the same order on every render.
2. **Only Call Hooks from React Functions:** Call them from React function components or custom Hooks. Do not call them from regular JavaScript functions.

1. State Hooks

State hooks let a component "remember" information like user input or current status between renders.

useState

Manages a reactive state variable in a function component.

- **Use Cases:** Simple values, toggles, form fields, and objects that do not have complex nested update logic.
- **How it works:** Returns the current state and a function to update it. Calling the updater triggers a re-render.

Code Example

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
```

```

<div className="card">
  <p>Count: {count}</p>
  {/ Use updater function when new state depends on previous state /}
  <button onClick={() => setCount(prev => prev + 1)}>Increment</button>
  <button onClick={() => setCount(0)}>Reset</button>
</div>
);
}

```

[!IMPORTANT] State updates are batched and asynchronous. If you need to update state based on the previous state, always pass an updater function (e.g., `setCount(prev => prev + 1)`) instead of the direct value (`setCount(count + 1)`).

useReducer

An alternative to `useState` for managing complex state structures or state updates that depend on multiple actions.

- **Use Cases:** Complex state objects, list management (todo lists, carts), and state transitions matching state machines.
- **How it works:** Takes a reducer function (`state, action`) => `newState` and initial state. Returns current state and a dispatch function.

Code Example

```

import { useReducer } from 'react';

const initialState = { count: 0, status: 'Idle' };

function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { ...state, count: state.count + 1, status: 'Incremented' };
    case 'decrement':

```

```

    return { ...state, count: state.count - 1, status: 'Decrement' };
  case 'reset':
    return initialState;
  default:
    throw new Error(`Unhandled action type: ${action.type}`);
  }
}

function CounterWithReducer() {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <div>
      <p>Count: {state.count} (Status: {state.status})</p>
      <button onClick={() => dispatch({ type: 'increment' })}>+</button>
      <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
      <button onClick={() => dispatch({ type: 'reset' })}>Reset</button>
    </div>
  );
}

export default app;

```

//dispatch is a function returned by the useReducer Hook that is used to send an action to the reducer function. The reducer processes the action and returns the updated state.

2. Context Hooks

useContext

Subscribes to a React context without nesting a Consumer component.

- **Use Cases:** Global themes, user authentication details, language/locale, and application settings.
- **How it works:** Takes a context object (created with `React.createContext`) and returns the current context value. When the provider value changes, the component automatically re-renders.

Code Example

```
import { createContext, useContext, useState } from 'react';

const ThemeContext = createContext(null);

export default function App() {
  const [theme, setTheme] = useState('dark');
  return (
    <ThemeContext.Provider value={theme}>
      <Card />
      <button onClick={() => setTheme(t => t === 'dark' ? 'light' : 'dark')}>
        Toggle Theme
      </button>
    </ThemeContext.Provider>
  );
}

function Card() {
  const theme = useContext(ThemeContext);
  return (
    <div className={`card ${theme}`}>
      <p>The current theme is {theme}</p>
    </div>
  );
}
```

[!WARNING] `useContext` will always trigger a re-render when the context value changes, even if the component only uses a small part of that context. To optimize, split contexts or memoize children.

3. Ref Hooks

Refs allow components to hold mutable values that do not trigger re-renders, and directly access DOM elements.

useRef

Creates a mutable reference object whose `.current` property is persisted across renders.

- **Use Cases:** Referencing DOM nodes (focusing input, scrolling), storing timers/intervals, or keeping track of the previous state value.
- **How it works:** Returns a ref object. Mutating `.current` does **not** cause a re-render.

Code Example

```
import { useRef, useEffect } from 'react';

function AutoFocusInput() {
  const inputRef = useRef(null);
  const timerRef = useRef(null);

  useEffect(() => {
    // Focus the input on mount
    inputRef.current.focus();

    timerRef.current = setInterval(() => {
      console.log('Tick');
    }, 1000);

    return () => clearInterval(timerRef.current);
  }, []);
```

```
return <input ref={inputRef} placeholder="Type here..." />;
}
```

useImperativeHandle

Customizes the instance value that is exposed to parent components when using ref.

- **Use Cases:** Creating reusable components (like custom modal boxes, media players) that expose custom imperative APIs (like `open()`, `close()`, `play()`).
- **How it works:** Must be used with `forwardRef`. Defines functions/values attached to the parent ref.

Code Example

```
import { useRef, useImperativeHandle, forwardRef } from 'react';

const CustomInput = forwardRef((props, ref) => {
  const localInputRef = useRef();

  useImperativeHandle(ref, () => ({
    focusAndSelect: () => {
      localInputRef.current.focus();
      localInputRef.current.select();
    }
  }));

  return <input ref={localInputRef} {...props} />;
});

// Parent component usage:
function Parent() {
  const inputRef = useRef();
  return (
    <>
      <CustomInput ref={inputRef} defaultValue="Hello" />
    </>
  );
}
```

```
<button onClick={() => inputRef.current.focusAndSelect()}>
  Focus & Select
</button>
</>
);
}
```

4. Effect Hooks

Effects let a component connect to and synchronize with external systems (network, browser DOM, third-party libraries).

useEffect

Synchronizes a component with an external system.

- **Use Cases:** API calls, subscribing to events (websocket, resize), timers, and manipulating the DOM imperatively.
- **How it works:** Runs after browser paint. Accepts a function and a dependency array. Returns an optional cleanup function.

Code Example

```
import { useState, useEffect } from 'react';

function FetchUser({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    let isCancelled = false;

    async function loadData() {
      const response = await fetch(`https://api.example.com/users/${userId}`);
      const data = await response.json();
      if (!isCancelled) {
        setUser(data);
      }
    }
  }, [userId]);
}
```

```

    }
    loadData();

    // Cleanup function: runs before next effect execution or unmount
    return () => {
      isCancelled = true;
    };
  }, [userId]); // Dependency array: triggers only when userId changes

  return user ? <div>{user.name}</div> : <p>Loading...</p>;
}

```

[!TIP] Always return a cleanup function to prevent memory leaks and race conditions (e.g., setting state on unmounted components or outdated requests).

useLayoutEffect

Fires synchronously after all DOM mutations but **before** the browser paints the screen.

- **Use Cases:** Measuring layouts (width, height, element position) and adjusting styling or scroll position before the user sees it (prevents flickering).
- **How it works:** Identical signature to `useEffect`, but runs blocking.

Code Example

```

import { useLayoutEffect, useRef, useState } from 'react';

function Tooltip({ children }) {
  const [height, setHeight] = useState(0);
  const containerRef = useRef(null);

  useLayoutEffect(() => {

```

```

// Measure DOM height before paint
if (containerRef.current) {
  setHeight(containerRef.current.getBoundingClientRect().height);
}
}, []);

return (
  <div ref={containerRef} style={{ bottom: `${height}px` }}>
    {children}
  </div>
);
}

```

useInsertionEffect

Fires before any DOM mutations and before `useLayoutEffect`. It is intended specifically for CSS-in-JS libraries.

- **Use Cases:** Dynamic styling in styling libraries (like styled-components) to insert `<style>` tags before layout calculations.
- **How it works:** Runs synchronously before DOM updates. Does not have access to refs.

5. Performance Hooks

Performance hooks optimize the rendering behavior by memoizing computations or deferring slow updates.

useMemo

Caches the result of a calculation between re-renders.

- **Use Cases:** Filtering lists, parsing large payloads, and preventing expensive derived-state calculations on every render.
- **How it works:** Computes the value only when dependencies change. Otherwise, returns the cached result.

Code Example

```
import { useMemo, useState } from 'react';

function ProductList({ items, filterText }) {
  const filteredItems = useMemo(() => {
    console.log('Filtering items...'); // Runs only if items or filterText change
    return items.filter(item => item.name.includes(filterText));
  }, [items, filterText]);

  return (
    <ul>
      {filteredItems.map(item => <li key={item.id}>{item.name}</li>)}
    </ul>
  );
}
```

useCallback

Caches a function definition between re-renders.

- **Use Cases:** Passing callback functions to optimized child components (React.memo) to prevent them from re-rendering due to reference changes.
- **How it works:** Returns the exact same function reference until dependencies change.

Code Example

```

import { useCallback, useState, memo } from 'react';

// Optimized child component
const Button = memo(({ onClick, label }) => {
  console.log(`Rendered: ${label}`);
  return <button onClick={onClick}>{label}</button>;
});

function Container() {
  const [count, setCount] = useState(0);

  // Reference does not change on count updates
  const handleAction = useCallback(() => {
    console.log('Button clicked');
  }, []);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(c => c + 1)}>Increment Count</button>
      <Button onClick={handleAction} label="Action Button" />
    </div>
  );
}

```

useTransition

Lets you update the state without blocking the UI.

- **Use Cases:** Tab switching, live search filters, and updates that generate heavy rendering workflows.
- **How it works:** Returns an `isPending` state and a `startTransition` function. State updates inside `startTransition` are marked as non-blocking transitions and can be interrupted by user actions like typing.

Code Example

```
import { useState, useTransition } from 'react';

function SearchResults() {
  const [query, setQuery] = useState("");
  const [list, setList] = useState([]);
  const [isPending, startTransition] = useTransition();

  const handleChange = (e) => {
    const value = e.target.value;
    setQuery(value); // High priority: update input instantly

    // Transition (Low priority): process list filter without locking input
    startTransition(() => {
      const items = Array.from({ length: 20000 }, (_, i) => `${value} item ${i}`);
      setList(items);
    });
  };

  return (
    <div>
      <input type="text" value={query} onChange={handleChange} />
      {isPending ? <p>Loading list...</p> : (
        <ul>
          {list.slice(0, 100).map((item, i) => <li key={i}>{item}</li>)}
        </ul>
      )}
    </div>
  );
}
```

useDeferredValue

Lets you defer updating a part of the UI. Similar to debounce/throttle but managed natively by React's scheduler.

- **Use Cases:** Slow, expensive lists that update as typing occurs.
- **How it works:** Returns a deferred version of a value that trails the original value. During a transition, React will try to reuse the old value and then update it in the background.

Code Example

```
import { useState, useDeferredValue, useMemo } from 'react';

function SearchDashboard() {
  const [text, setText] = useState("");
  const deferredText = useDeferredValue(text);

  // Computed using deferred value: input response remains immediate
  const list = useMemo(() => {
    return Array.from({ length: 10000 }, (_, i) => `${deferredText} item ${i}`);
  }, [deferredText]);

  return (
    <div>
      <input value={text} onChange={e => setText(e.target.value)} />
      <ExpensiveList list={list} />
    </div>
  );
}

function ExpensiveList({ list }) {
  return (
    <ul>
      {list.slice(0, 50).map((item, index) => <li key={index}>{item}</li>)}
    </ul>
  );
}
```

6. Resource and Form Hooks (React 19)

React 19 introduces hooks specifically designed for actions, resources, and async state mutations.

use

Reads resources (like Promises or Context) inline inside conditional loops and checks.

- **Use Cases:** Resolving async data directly in rendering (suspends the component), or loading conditional context values.
- **How it works:** Unlike other hooks, use can be called inside loops and if statements.

Code Example

```
import { use, Suspense } from 'react';

const fetchMessage = new Promise((resolve) => setTimeout(() => resolve('Hello from React 19!'), 1000));

function MessageComponent() {
  // Resolves the promise. Component suspends until resolved.
  const message = use(fetchMessage);
  return <h2>{message}</h2>;
}

export default function App() {
  return (
    <Suspense fallback={<p>Loading message...</p>}>
      <MessageComponent />
    </Suspense>
  );
}
```

useActionState (React 19 / formerly useFormState)

Hook designed to simplify forms using Actions (asynchronous function submission).

- **Use Cases:** Form submission handling, resolving validation errors, and display of pending/completed states.
- **How it works:** Accepts an action function and initial state. Returns the current state, a wrapped action function (for form actions), and an isPending indicator.

Code Example

```
import { useActionState } from 'react';

async function updateUsername(prevState, formData) {
  const username = formData.get('username');
  try {
    // API mock call
    await new Promise((resolve) => setTimeout(resolve, 1000));
    return { success: true, message: `Username updated to ${username}` };
  } catch (err) {
    return { success: false, message: 'Failed to update username' };
  }
}

function ProfileForm() {
  const [state, formAction, isPending] = useActionState(updateUsername, null);

  return (
    <form action={formAction}>
      <input type="text" name="username" required />
      <button type="submit" disabled={isPending}>
        {isPending ? 'Updating...' : 'Update'}
      </button>
      {state && <p className={state.success ? 'success' :
'error'}>{state.message}</p>}
    </form>
  )
}
```

```
);  
}
```

useFormStatus (React 19)

Retrieves submission status of a parent <form> container.

- **Use Cases:** Creating nested child action components (like specialized Submit buttons) that need to disable themselves during parents' async posts.
- **How it works:** Read state parameters from the nearest parent form. Must be called in a child component inside the <form>.

Code Example

```
import { useFormStatus } from 'react-dom';  
  
function SubmitButton() {  
  const { pending, data, method, action } = useFormStatus();  
  
  return (  
    <button type="submit" disabled={pending}>  
      {pending ? 'Saving...' : 'Save Changes'}  
    </button>  
  );  
}  
  
// Parent component:  
function FormWrapper() {  
  return (  
    <form action={async () => { / async action / }}>  
      <input type="text" name="input" />  
      <SubmitButton /> {/ Hook is called here inside child /}  
    </form>  
  );  
}
```

useOptimistic (React 19)

Displays a temporary state value during async actions, reverting it if the action fails.

- **Use Cases:** Message bubbles, likes/ratings, checklist completions where UI updates instantaneously before API verification completes.
- **How it works:** Takes original state and a reducer-like function. Returns optimistic state value and an updater function.

Code Example

```
import { useOptimistic, useState } from 'react';

function MessageBoard() {
  const [messages, setMessages] = useState([ { text: 'Hello!' } ]);
  const [optimisticMessages, addOptimisticMessage] = useOptimistic(
    messages,
    (state, newMessageText) => [...state, { text: newMessageText, sending: true } ]
  );

  async function sendMessageAction(formData) {
    const messageText = formData.get('message');

    // 1. Immediately apply optimistic UI change
    addOptimisticMessage(messageText);

    // 2. Perform background server action
    await new Promise((resolve) => setTimeout(resolve, 1500)); // Mock API delay

    // 3. Confirm true state update
    setMessages(prev => [...prev, { text: messageText }]);
  }

  return (
```

```

<div>
  {optimisticMessages.map((m, i) => (
    <p key={i} style={{ opacity: m.sending ? 0.5 : 1 }}>
      {m.text} {m.sending && '(sending...)}'
    </p>
  ))}
  <form action={sendMessageAction}>
    <input type="text" name="message" required />
    <button type="submit">Send</button>
  </form>
</div>
);
}

```

7. Miscellaneous Hooks

useId

Generates unique, stable IDs that are compatible with server-side rendering (SSR) and client hydration.

- **Use Cases:** Generating unique HTML element IDs for accessibility bindings (e.g. aria-describedby or labels to inputs).

Code Example

```

import { useId } from 'react';

function FormField() {
  const id = useId();
  return (
    <div>
      <label htmlFor={id}>Username:</label>
      <input id={id} type="text" />
    </div>
  );
}

```

```
);  
}
```

useSyncExternalStore

Subscribes to an external store (like Redux, Zustand, router paths, or browser status API).

- **Use Cases:** Synchronizing non-React libraries or browser APIs (like network status, storage status) to component rendering without tear.
- **How it works:** Takes subscribe function, getSnapshot function, and an optional getServerSnapshot function.

Code Example

```
import { useSyncExternalStore } from 'react';  
  
// Subscribe to browser online/offline status  
function getSnapshot() {  
  return navigator.onLine;  
}  
  
function subscribe(callback) {  
  window.addEventListener('online', callback);  
  window.addEventListener('offline', callback);  
  return () => {  
    window.removeEventListener('online', callback);  
    window.removeEventListener('offline', callback);  
  };  
}  
  
function OnlineIndicator() {  
  const isOnline = useSyncExternalStore(subscribe, getSnapshot);  
  return <p>Status: {isOnline ? '🟢 Online' : '🔴 Offline'}</p>;  
}
```

useDebugValue

Displays custom debug labels in React Developer Tools.

- **Use Cases:** Used within custom Hooks to show developer-facing labels next to Hook states.

Code Example

```
import { useState, useDebugValue } from 'react';

function useFriendStatus(friendID) {
  const [isOnline, setIsOnline] = useState(null);

  // Displays online/offline tag inside React DevTools
  useDebugValue(isOnline ? 'Online' : 'Offline');

  return isOnline;
}
```

Practical Scenarios & Questions for Each Hook

Below are typical development scenarios and interview questions mapped to each hook to help you identify when to use them.

`useState`

- Question: "How do I track form inputs (e.g., name, email, password) dynamically as a user types, and reset them on submit?"
- Solution: Maintain an object state like `const [form, setForm] = useState({ name: "", email: "", password: "" })` and update the respective keys using input `onChange` handlers.

`useReducer`

- Question: "I am building a shopping cart where users can add items, remove items, increase/decrease item quantities, and apply promo codes. My `useState` setters are becoming huge and messy. How can I clean this up?"
- Solution: Implement a central reducer function that processes actions (e.g., `dispatch({ type: 'ADD_ITEM', payload: item })`) and manage it via `useReducer`.

`useContext`

- Question: "I have a global settings configuration (like light/dark mode theme or user authentication status). How can I make it available to deeply nested components without passing props through every intermediate child?"
- Solution: Create a custom context and wrap your tree with a provider, then consume it inside components using `const theme = useContext(ThemeContext)`.

`useRef`

- Question: "How do I focus a text input immediately when a modal opens, or how can I track how many times a component has rendered without triggering a re-render itself?"
- Solution: Store the reference to the DOM element or a counter in a ref: `const countRef = useRef(0)`. Mutate `.current` directly.

`useImperativeHandle`

- Question: "I am writing a reusable custom drawer/modal component. How do I expose imperative methods like `.open()` and `.close()` to the parent component using a ref?"
- Solution: Combine `forwardRef` with `useImperativeHandle` inside the modal component to expose custom callbacks.

`useEffect`

- Question: "How do I load a user profile from an external database API when

the user's ID changes, and ensure old, lagging network calls do not overwrite newer ones?"

- Solution: Trigger the fetch inside `useEffect` with `[userId]` as a dependency, and return a cleanup function to toggle an `active` boolean flag to ignore stale responses.

`useLayoutEffect`

- Question: "I am building a tooltip component. When it opens, it measures the anchor button's dimensions to position itself. In standard `useEffect`, the tooltip appears in the wrong spot for a brief frame and then jumps. How do I prevent this flicker?"
- Solution: Calculate positions synchronously using `useLayoutEffect` before the browser paints the screen.

`useInsertionEffect`

- Question: "I am authoring a custom styling library. How do I dynamically inject `<style>` rules into the DOM before layout calculations occur, minimizing browser style recalculation costs?"
- Solution: Use `useInsertionEffect` to append stylesheet rules onto the document before rendering begins.

`useMemo`

- Question: "My component receives a large list of 5,000 logs and filters them. Typing in a totally unrelated text box causes the list filtering to re-run, causing lag. How do I prevent this?"
- Solution: Wrap the filtering logic in `useMemo` so it recalculates only when either the log list or the filter query changes.

`useCallback`

- Question: "I have a parent component that re-renders. I pass an `onRemove` callback to list items optimized with `React.memo`. Why do the child list items still re-render on every parent render?"
- Solution: Because a new function reference for `onRemove` is created on every parent render. Wrap the function in `useCallback` to maintain reference equality.

`useTransition`

- Question: "I have a live search that filters an extensive database. As I type, the input box is unresponsive and lags behind. How do I keep input typing fast while generating search updates?"

- Solution: Treat the input text state update as high priority, and wrap the database query/list state update in `startTransition` to calculate results in the background.

`useDeferredValue`

- Question: "I am passing a search query down to an expensive third-party graphing component that I can't optimize. How can I defer updating the graph until the user pauses typing?"

- Solution: Create a deferred version of the query: `const deferredQuery = useDeferredValue(query)`, and feed the deferred value into the graph component.

`use`

- Question: "How do I conditionally consume context values or fetch data directly inside nested loops or `if` statements in React 19?"

- Solution: Use the `use(Resource)` hook. It resolves promises or context values and can be called conditionally, unlike traditional React hooks.

`useActionState`

- Question: "How do I handle a form submission that calls an asynchronous API, automatically tracking when it's pending (loading state) and storing any validation errors returned by the server?"

- Solution: Bind the form to an Action using `const [state, formAction, isPending] = useActionState(mySubmitAction)`.

`useFormStatus`

- Question: "I have a reusable submit button component nested inside a complex form. How can this button know if the parent form is currently submitting, without passing loading props down?"

- Solution: Call `const { pending } = useFormStatus()` inside the button component. It will auto-detect the status of the closest parent form.

`useOptimistic`

- Question: "How do I implement an instant 'Like' button or messaging bubble

that updates immediately in the UI while the background server request is still running?"

- Solution: Use `useOptimistic` to show the expected successful state instantly. If the action succeeds, state updates permanently; if it fails, it rolls back.

`useId`

- Question: "I am building accessible form components. How can I match `<label htmlFor={...}>` with an input's `<id>` ensuring that the generated ID matches between the server and the client during hydration?"
- Solution: Call `const id = useId()` to generate a stable, collision-free identifier.

`useSyncExternalStore`

- Question: "I am subscribing to a custom event bus, window width, or Redux store. How do I read values from it dynamically in React without risking 'tearing' (inconsistent UI) during concurrent rendering?"
- Solution: Register the store and retrieve snapshots via `useSyncExternalStore(subscribe, getSnapshot)`.

`useDebugValue`

- Question: "I am creating a custom hook named `useGeolocation`. How do I format and display its internal state values (like 'Lat: 45.2, Lon: 12.3') inside the React Developer Tools extension?"
- Solution: Add `useDebugValue(location ? `Lat: ${location.lat}` : 'Loading')` inside your custom hook.

CustomHooks

Custom React Hooks

React's **hooks** let you use state and other React features without writing a class. While the builtin hooks (useState, useEffect, useContext, ...) cover most scenarios, realworld projects often need **reusable logic** that spans multiple components. That's where **custom hooks** shine.

Why do we need custom hooks?

Reason	Explanation
Encapsulation of reusable logic	Pull out repeated patterns (e.g., data fetching, form handling, debouncing) into a single, testable unit.
Cleaner component code	Components stay focused on UI, while the hook handles the "how".
Shareable across projects	Export a hook as a library and reuse it in any React app.
Leverages existing hooks	Custom hooks are just functions that can call other hooks, giving you the full power of the React hook ecosystem.

When to create a custom hook?

1. **Multiple components need the same stateful logic** (e.g., pagination, infinite scroll).
2. **Complex sideeffects** that would clutter a component (e.g., WebSocket connections, subscriptions).
3. **Domainspecific abstractions** – turning a business concept into an API (e.g., useAuth, useCart).

4. **Testing** – isolating logic makes unit testing straightforward.

Anatomy of a custom hook

A custom hook is simply a JavaScript function whose name **must start with use** and that may call **any builtin or other custom hooks**.

```
// Valid custom hook
function useMyFeature(arg) {
  // you can call other hooks here
  const [state, setState] = React.useState(null);
  // ...logic using args, state, effects, etc.
  return state;
}
```

Key rules

- Must start with use (enforced by the linter and React's Rules of Hooks).
- Must be called **only at the top level** of a React function component or another hook.
- Do **not** call hooks conditionally inside loops or nested functions.

Common usecase examples

-> Data fetching hook (useFetch)

```
import { useState, useEffect } from "react";

/**
 * Fetches JSON data from a URL and returns loading / error states.
 */
export function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let isMounted = true; // avoid state updates after unmount
    setLoading(true);
    fetch(url)
      .then((res) => {
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        return res.json();
      })
      .catch((err) => {
        setError(err);
        if (isMounted) setLoading(false);
      });
  });
}
```

```

    })
    .then((json) => {
      if (isMounted) setData(json);
    })
    .catch((err) => {
      if (isMounted) setError(err);
    })
    .finally(() => {
      if (isMounted) setLoading(false);
    });
  return () => {
    isMounted = false;
  };
}, [url]); // refetch when URL changes

return { data, loading, error };
}

```

Usage

```

function UsersList() {
  const { data: users, loading, error } = useFetch("/api/users");

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error.message}</p>;
  return (
    <ul>
      {users.map((u) => (
        <li key={u.id}>{u.name}</li>
      ))}
    </ul>
  );
}

```

-> Form handling hook (useForm)

```

import { useState } from "react";

export function useForm(initialValues) {
  const [values, setValues] = useState(initialValues);

  const handleChange = (e) => {
    const { name, value, type, checked } = e.target;
    setValues((prev) => ({
      ...prev,
      [name]: type === "checkbox" ? checked : value,
    }));
  };
}

```

```

};

const reset = () => setValues(initialValues);

const handleSubmit = (onSubmit) => (e) => {
  e.preventDefault();
  onSubmit(values);
};

return { values, handleChange, reset, handleSubmit };
}

```

Usage

```

function ContactForm() {
  const { values, handleChange, reset, handleSubmit } = useForm({
    name: "",
    email: "",
    message: "",
  });

  const submit = (data) => {
    console.log("Submitting", data);
    // ...send to API
    reset();
  };

  return (
    <form onSubmit={handleSubmit(submit)}>
      <input name="name" value={values.name} onChange={handleChange} placeholder="Name" />
      <input name="email" type="email" value={values.email} onChange={handleChange}
placeholder="Email" />
      <textarea name="message" value={values.message} onChange={handleChange} />
      <button type="submit">Send</button>
    </form>
  );
}

```

-> Debounce hook (useDebounce)

```

import { useState, useEffect } from "react";

/**
 * Returns a debounced version of a value.
 * Useful for search inputs, resize listeners, etc.
 */
export function useDebounce(value, delay = 300) {

```

```
const [debounced, setDebounced] = useState(value);

useEffect(() => {
  const handler = setTimeout(() => setDebounced(value), delay);
  return () => clearTimeout(handler);
}, [value, delay]);

return debounced;
}
```

Usage

```
function SearchBox() {
  const [query, setQuery] = useState("");
  const debouncedQuery = useDebounce(query, 500);

  // fetch only when debouncedQuery changes
  const { data, loading } = useFetch(`/api/search?q=${debouncedQuery}`);

  return (
    <input
      placeholder="Search..."
      value={query}
      onChange={(e) => setQuery(e.target.value)}
    />
  );
}
```

Spread and Rest

The Spread and Rest Operators in JavaScript (...)

In JavaScript (and React), the three dots syntax ... performs two opposite operations depending on **where** and **how** it is used: **Spread** and **Rest**.

1. Quick Summary

- **Spread Operator:** Unpacks elements (spreads them out). It takes an array or object and breaks it down into individual components.
- **Rest Parameter:** Packs elements (gathers them up). It collects multiple individual elements and condenses them into a single array or object.

2. The Spread Operator (Unpacking)

Spread is used to duplicate, merge, or convert collections into separate items.

A. Spread with Arrays

```
const fruits = ['apple', 'banana'];
const veggies = ['brocoli', 'carrot'];

// 1. Copying an array
const fruitsCopy = [...fruits]; // ['apple', 'banana']

// 2. Concatenating / Merging arrays
const food = [...fruits, ...veggies, 'pizza'];
// Result: ['apple', 'banana', 'brocoli', 'carrot', 'pizza']

// 3. Passing array items as separate arguments to a function
const numbers = [5, 12, 8];
const max = Math.max(...numbers); // Equivalent to Math.max(5, 12, 8) -> returns 12
```

B. Spread with Objects (Crucial for React State)

```
const user = { name: 'Sarah', role: 'Admin' };

// 1. Copying and adding/modifying properties
const updatedUser = { ...user, age: 25, role: 'Superadmin' };
// Result: { name: 'Sarah', role: 'Superadmin', age: 25 }
```

```
// 2. Merging two objects
const details = { city: 'Boston' };
const fullProfile = { ...user, ...details };
// Result: { name: 'Sarah', role: 'Admin', city: 'Boston' }
```

[!WARNING] Both Array and Object Spread perform a **shallow copy**. If the original object contains nested objects/arrays, they are copied by **reference** (not deep value). Changing nested objects in the copy will affect the original.

```
const user = { name: 'Alex', info: { age: 20 } };
const copy = { ...user };
copy.info.age = 21; // Modifies BOTH user and copy because copy.info is still pointing to the same nested address!
```

3. The Rest Operator (Packing / Gathering)

Rest gathers multiple separate values into a single compound structure (an array or object).

A. Rest as Function Parameters

When defined in a function's argument list, it collects all remaining arguments into a clean array.

```
// Gathers any number of numbers into a 'nums' array
function sumEverything(multiplier, ...nums) {
  // nums = [1, 2, 3]
  return nums.reduce((total, n) => total + n, 0) * multiplier;
}

console.log(sumEverything(10, 1, 2, 3));
// Output: (1 + 2 + 3) * 10 = 60
```

*Note: The rest parameter must always be the **last** parameter in the function signature.*

B. Rest in Array Destructuring

```
const colors = ['red', 'green', 'blue', 'yellow'];

// Extract the first color, and gather the rest
const [primary, ...remainingColors] = colors;

console.log(primary);      // 'red'
console.log(remainingColors); // ['green', 'blue', 'yellow']
```

C. Rest in Object Destructuring (Often used in React props)

```
const buttonConfig = { label: 'Submit', color: 'blue', size: 'large', disabled: false };

// Destructure the label, and collect all other style/settings configs
const { label, ...styleProps } = buttonConfig;

console.log(label);    // 'Submit'
console.log(styleProps); // { color: 'blue', size: 'large', disabled: false }
```

4. Visual Comparison for Students

You can visualize the operations like this:

Spread (Unpacking Box)

```
Array: [ 'A', 'B' ]
      \ /
Spread:  A , B    (Individual elements released)
```

Rest (Packing Items into Box)

```
Elements:  A , B
          \ /
Rest:      [ 'A', 'B' ] (Collected into a container)
```

React Props

React Props

1. What Are Props?

- **Props** (short for *properties*) are readonly values passed from a parent component to a child component.
- They are the **only** way to make a component dynamic and reusable.
- In JSX a prop looks like an HTML attribute:

```
<Child name="Alex" age={30} />
```

- Inside Child the values are received via the function argument:

```
function Child({ name, age }) {  
  return <p>{name} is {age} years old.</p>;  
}
```

2. Basic Syntax & Destructuring

```
// Parent.jsx  
import Child from './Child';  
  
export default function Parent() {  
  const user = { name: 'ayush', age: 21, section: 'A' };  
  return <Child {...user} />;  
}  
  
// Child.jsx  
export default function Child({ name, age }) {  
  return (  
    <div>  
      <h2>{name}</h2>  
      <p>Age: {age}</p>  
    </div>  
  );  
}
```

- Using **object spreading** (`{...user}`) is a concise way to pass many props at once.
- Destructuring in the function signature keeps the component tidy and avoids props.name repetition.

3. Default Props

When a parent does not provide a prop, you can give it a fallback value.

3.1 With Function Default Parameters (recommended in modern React)

```
function Greeting({ message = 'Hello, world!', name = 'Guest' }) {  
  return <p>{message} – {name}</p>;  
}
```

3.2 Using defaultProps (legacy, works with class components)

```
function Button({ label }) {  
  return <button>{label}</button>;  
}
```

```
Button.defaultProps = {  
  label: 'Click me',  
};
```

Note: For functional components, default parameters are clearer and typefriendly.

4. Children – The Special props.children

children is a builtin prop that contains whatever you nest between a component's opening and closing tags.

```
function Card({ title, children }) {  
  return (  
    <div className="card">  
      <h3>{title}</h3>  
      <div className="content">{children}</div>  
    </div>  
  );  
}
```

```
// Usage  
<Card title="Profile">  
  <p>Some descriptive text.</p>  
    
</Card>
```

- children can be a single node, an array of nodes, or even a render function.

5. Prop Types – Runtime Validation (JavaScript)

React ships with the prop-types package for declarative type checking.

```
npm install prop-types

import PropTypes from 'prop-types';

function Profile({ name, age, isAdmin, onLogout }) {
  return (
    <div>
      <h2>{name}</h2>
      <p>{age} years old</p>
      {isAdmin && <button onClick={onLogout}>Logout</button>}
    </div>
  );
}

Profile.propTypes = {
  name: PropTypes.string.isRequired,
  age: PropTypes.number,
  isAdmin: PropTypes.bool,
  onLogout: PropTypes.func,
};

Profile.defaultProps = {
  age: 0,
  isAdmin: false,
  onLogout: () => {},
};
```

- `isRequired` raises a console warning if the prop is missing.
- Helpful during development, but strips out in production builds.

6. Props with TypeScript – Compiletime Safety

If your project uses TypeScript, define a Props interface or type.

```
import React from 'react';

type ButtonProps = {
  label: string;
  disabled?: boolean;
  onClick: (event: React.MouseEvent<HTMLButtonElement>) => void;
```

```
};

const Button: React.FC<ButtonProps> = ({ label, disabled = false, onClick }) => (
  <button disabled={disabled} onClick={onClick}>
    {label}
  </button>
);

export default Button;
```

- Optional props are marked with ?.
- Default values can still be set via function defaults (disabled = false).
- TypeScript will catch mismatched prop names **before** the code even runs.

7. Passing Callback Functions (Event Handlers)

A common pattern is to give a child a function it can call back to the parent.

```
// Parent.jsx
function Parent() {
  const [count, setCount] = useState(0);

  const increment = () => setCount(c => c + 1);

  return (
    <div>
      <h1>Count: {count}</h1>
      <CounterButton onClick={increment} />
    </div>
  );
}

// CounterButton.jsx
function CounterButton({ onClick }) {
  return <button onClick={onClick}>+1</button>;
}
```

- The parent owns the state, the child only triggers an action.
- **Never modify a parent's state directly inside a child** – always go through a callback.

8. Rest Props (...rest) – Forwarding Unknown Props

When you want a component to accept *any* extra attributes (e.g., `className`, `data-*`, `aria-*`), collect them with rest syntax and spread them onto a DOM element.

```
function Input({ label, ...rest }) {  
  return (  
    <label>  
      {label}  
    <input {...rest} />  
    </label>  
  );  
}
```

```
// Usage – all extra props are passed to the <input>  
<Input label="Email" type="email" placeholder="you@example.com" required />
```

- This pattern makes components flexible and integrates nicely with libraries like **styledcomponents** or **MUI**.

9. Advanced Pattern – Render Props

A *render prop* is a function prop that returns JSX. It lets a component decide **what** to render while the parent supplies **how**.

```
function DataFetcher({ url, children }) {  
  const [data, setData] = useState(null);  
  const [loading, setLoading] = useState(true);  
  
  useEffect(() => {  
    fetch(url)  
      .then(r => r.json())  
      .then(json => {  
        setData(json);  
        setLoading(false);  
      });  
  }, [url]);  
  
  return children({ data, loading }); // <-- children is a function  
}  
  
// Usage  
<DataFetcher url="/api/user">  
  {({ data, loading }) =>  
    loading ? <p>Loading...</p> : <pre>{JSON.stringify(data, null, 2)}</pre>  
  }  
</DataFetcher>
```

- This is a flexible alternative to higherorder components (HOC).

10. Common Pitfalls & Best Practices

Pitfall	Why It's Bad	Fix
Mutating prop objects (props.user.age = 30)	Props are readonly; mutating breaks the contract and can cause stale UI.	Treat props as immutable; derive new objects with spread ({...props.user, age: 30}).
Passing new function objects on every render (onClick={() => doSomething()}))	Triggers unnecessary rerenders of child components using React.memo.	Define callbacks with useCallback or move them outside the render.
Forgetting key when rendering list of children	React cannot reconcile items correctly, leading to bugs.	Provide a stable key (key={item.id}).
Using defaultProps with functional components	Default parameters are clearer; defaultProps may be stripped in future TSX.	
Overusing ...rest on a component that also defines explicit props	Can unintentionally forward unwanted attributes to the DOM, causing warnings.	
Missing PropTypes/TS definitions	Harder to debug and no autocompletion.	

Passing primitive values that change often (e.g., objects created inline)	Causes rerenders because the reference changes each render.	Memoise objects with <code>useMemo</code> .
--	---	---

Recap

- Props are immutable inputs from parent to child.
- Use destructuring, default values, and **spread** (...) for clean APIs.
- Validate with **PropTypes** or **TypeScript**.
- Pass callbacks for child to parent communication.
- Leverage children, render props, and rest props for advanced composition.
- Follow best practice guidelines to avoid common bugs and keep your UI performant.

Prop Drilling, ContextAPI

1. What is Prop Drilling?

Prop drilling happens when you pass data from a parent component down to a deeply nested child component through **intermediate components that don't actually need that data themselves** — they only accept it and forward it further down.

Example of Prop Drilling (Theme Toggle)

None

```
// App.jsx (Parent)
import React, { useState } from "react";
import { ChildA } from "../ChildA";

export const App = () => {
  const [theme, setTheme] = useState("light");

  return (
    <div>
      <h1>Current theme: {theme}</h1>
      {/* Pass theme and setter down the tree */}
      <ChildA theme={theme} setTheme={setTheme} />
    </div>
  );
};
```

None

```
// ChildA.jsx (intermediate component)
import React from "react";
import { ChildB } from "../ChildB";

export const ChildA = ({ theme, setTheme }) => (
  <section>
    {/* This component does not use the props, it merely forwards them */}
    <ChildB theme={theme} setTheme={setTheme} />
  </section>
);
```

None

```
// ChildB.jsx (intermediate component)
import React from "react";
import { GrandChild } from "../GrandChild";

export const ChildB = ({ theme, setTheme }) => (
  <div>
    {/* Still no direct usage – forwarding continues */}
    <GrandChild theme={theme} setTheme={setTheme} />
  </div>
);
```

None

```
// GrandChild.jsx (the real consumer)
import React from "react";

export const GrandChild = ({ theme, setTheme }) => (
  <button
    onClick={() => setTheme(theme === "light" ? "dark" : "light")}
    style={{
      background: theme === "light" ? "#fff" : "#333",
      color: theme === "light" ? "#000" : "#fff",
    }}
  >
    Toggle Theme (currently {theme})
  </button>
);
```

Here, theme and setTheme travel through ChildA → ChildB even though neither component uses them. They exist only as a delivery pipeline so GrandChild can eventually read and update the theme.

2. Problems Caused by Prop Drilling

Problem	Explanation
Unnecessary re-renders	ChildA and ChildB re-render whenever theme changes, even though they never use it.
Tight coupling	ChildA and ChildB are now forced to accept theme and setTheme in their props — they can't be reused elsewhere without also wiring these up.
Poor maintainability	If we add a new shared value (e.g., fontSize), we must update the prop signature of ChildA, ChildB, <i>and</i> GrandChild.
Harder refactoring	Renaming theme to mode means touching every file in the chain, not just GrandChild.
Reduced readability	Anyone reading ChildA or ChildB sees theme/setTheme in the props and has to dig further down to find out who actually uses them.
Scales badly	With just one intermediate layer it's mildly annoying; with five or six, it becomes very hard to trace.

3. Common Ways to Handle Prop Drilling

1. **Component composition** — pass JSX as children instead of threading data through multiple layers (works in some cases, not all).
2. **State management libraries** — Redux, Zustand, Recoil (heavier, useful for large/complex apps).
3. **React Context API** — a **built-in** solution for sharing state across the tree without manually passing props at every level. This is the best fix for our ChildA → ChildB → GrandChild problem.

We'll now rebuild the exact same example using Context API.

4. Introducing the Context API

The Context API lets you create a shared store of data that any component in the tree can read directly — **skipping all the intermediate levels**.

It has 3 core parts:

1. **createContext()** — creates the Context object.
2. **Provider** — supplies the value to every component below it in the tree.
3. **useContext()** (or the older Consumer) — reads the value from the nearest Provider above it.

Mental Model

Think of Context like a **WiFi router**:

- `createContext()` sets up the router.
 - `<Provider>` broadcasts the signal (the data) to everything inside its range (the component tree).
 - Any component that wants to "connect" just calls `useContext()` — no need to run a cable (props) through every room (component like `ChildA`, `ChildB`).
-

5. Step-by-Step Implementation (Same Example, Using Context)

Step 1 — Create the Context

None

```
// ThemeContext.jsx
import React, { createContext } from "react";

// createContext(defaultValue) - defaultValue is used only if no
// Provider wraps the component
const ThemeContext = createContext(null);
export default ThemeContext ;
```

`createContext(defaultValue)` returns a Context **object**. We'll fill in the real value using a `Provider` in the next step.

Step 2 — Wrap Your Tree with the Provider

The `Provider` accepts a `value` prop — the data you want to share. Any component nested inside it, no matter how deep, can read this value directly.

None

```
// App.jsx (Parent)
import React, { useState } from "react";
import { ThemeContext } from "../ThemeContext";
import { ChildA } from "../ChildA";

export const App = () => {
  const [theme, setTheme] = useContext("light");

  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <div>
        <h1>Current theme: {theme}</h1>
        { /* No more theme/setTheme props being passed! */ }
        <ChildA />
      </div>
    </ThemeContext.Provider>
  );
};
```

Notice: **ChildA no longer receives theme or setTheme as props at all.**

None

```
// ChildA.jsx (intermediate component)
import React from "react";
import { ChildB } from "../ChildB";

export const ChildA = () => (
  <section>
    { /* ChildA is now completely clean – no props to forward */ }
    <ChildB />
  </section>
);
```

None

```
// ChildB.jsx (intermediate component)
import React from "react";
import { GrandChild } from "../GrandChild";

export const ChildB = () => (
  <div>
    { /* ChildB is clean too */ }
    <GrandChild />
  </div>
);
```

Both ChildA and ChildB are now free of theme/setTheme — they don't know or care that theming exists.

Step 3 — Consume the Data

A) Using useContext (Recommended)

None

```
// GrandChild.jsx (the real consumer)
import React, { useContext } from "react";
import { ThemeContext } from "../ThemeContext";

export const GrandChild = () => {
  // useContext(Context) reads the nearest Provider's value
  // directly
  const { theme, setTheme } = useContext(ThemeContext);

  return (
    <button
      onClick={() => setTheme(theme === "light" ? "dark" :
"light")}
      style={{
        background: theme === "light" ? "#fff" : "#333",
        color: theme === "light" ? "#000" : "#fff",
      }}
    />
  );
};
```

```

    }}
  >
    Toggle Theme (currently {theme})
  </button>
);
};

```

`useContext(ThemeContext)` looks up the component tree for the **nearest** `<ThemeContext.Provider>` and returns its value. `GrandChild` automatically re-renders whenever theme changes — and `ChildA/ChildB` are untouched by this data entirely.

B) Using `<Context.Consumer>` (Older, still valid in class components)

None

```

// GrandChild.jsx (the real consumer)
import React from "react";
import { ThemeContext } from "../ThemeContext";

export const GrandChild = () => (
  <ThemeContext.Consumer>
    ({ { theme, setTheme } }) => (
      <button
        onClick={() => setTheme(theme === "light" ? "dark" :
"light")}
        style={{
          background: theme === "light" ? "#fff" : "#333",
          color: theme === "light" ? "#000" : "#fff",
        }}
      >
        Toggle Theme (currently {theme})
      </button>
    )}
  </ThemeContext.Consumer>
);

```

Consumer uses the **render props pattern** — a function as its child that receives the context value. This was the only option before Hooks (pre-React 16.8), and it's still used inside class components where Hooks can't be called.

Data flow: App provides theme / setTheme via ThemeContext.Provider → GrandChild grabs it directly via useContext, completely bypassing ChildA and ChildB.

6. Best Practice: Custom Hook for Context

Wrap useContext in a custom hook so consumers don't need to import both useContext and ThemeContext every time — and so misuse fails loudly instead of silently.

None

```
// ThemeContext.jsx
import React, { createContext, useContext } from "react";

export const ThemeContext = createContext(undefined);

// Custom hook
export const useTheme = () => {
  const context = useContext(ThemeContext);
  if (context === undefined) {
    throw new Error("useTheme must be used within a ThemeContext.Provider");
  }
  return context;
};
```

None

```
// GrandChild.jsx (the real consumer)
import React from "react";
import { useTheme } from "../ThemeContext";

export const GrandChild = () => {
  const { theme, setTheme } = useTheme(); // cleaner import,
  built-in safety check
```

```
return (  
  <button onClick={() => setTheme(theme === "light" ? "dark" :  
"light")}>  
    Toggle Theme (currently {theme})  
  </button>  
);  
};
```

This pattern makes misuse (calling `useTheme()` outside a `ThemeContext.Provider`) fail immediately with a clear error, instead of quietly returning `undefined`.

8. When to Use Context API vs. When Not To

Good use cases:

- Theme (light/dark mode) — exactly as shown above
- Authenticated user info
- Language/locale preference
- Global UI state (e.g., sidebar open/closed)

Avoid Context for:

- **High-frequency updates** (e.g., mouse position, animation values) — every consumer re-renders on every change, which can hurt performance.
 - **Very large, complex, frequently-changing global state** — prefer Redux, Zustand, or Recoil, which offer selective subscriptions and better performance optimization.
-

9. Summary Table

Concept	Purpose
Prop Drilling	Passing theme/setTheme through ChildA and ChildB, which don't need them, just to reach GrandChild.

Problem	Unnecessary re-renders, tight coupling, poor maintainability, hard to scale.
createContext()	Creates the ThemeContext object to hold shared theme data.
Provider	<code><ThemeContext.Provider value={{ theme, setTheme }}></code> wraps the tree and supplies data to all descendants.
Consumer / useContext	Reads the value from the nearest Provider; useContext is the modern, preferred approach.
Custom Hook Pattern	<code>useTheme()</code> wraps <code>useContext</code> for cleaner imports and safety checks.
Limitation	Not ideal for very frequent updates or extremely large global state — consider dedicated state libraries instead.

useMemo and React.Memo

What is Memoization?

Memoization is an **optimization technique** that caches the result of a *pure* (no sideeffects) function call so that subsequent calls with the **same inputs** can return the cached value **instantly** instead of recomputing it.

- **Pure function** – given the same arguments, it always returns the same output and does not depend on external mutable state.
- **Cache** – a data structure (often a simple variable) that stores the computed value.
- **Goal** – reduce expensive calculations, avoid unnecessary rerenders, and keep UI snappy.

In JavaScript, memoization can be implemented manually (e.g., with a Map) or, in the context of React, the framework provides two dedicated APIs:

API	Typical UseCase	What it memoizes
useMemo	Inside a component to memoize computed values (numbers, objects, arrays) that are expensive to derive.	The return value of a function passed to useMemo.
React.memo (or memo)	Wrap a function component to skip rendering when its props have not changed.	The entire rendered output of the component.

useMemo

Syntax

```
const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b]);
```

- The **factory function** runs *only* when one of the dependencies in the dependency array changes.
- If the dependency array is [], the function runs **once** on the first render (similar to componentDidMount).

Why use it?

1. **Heavy calculations** – loops, sorting, recursive algorithms, etc.
2. **Reference stability** – objects/arrays returned from a function will keep the same reference across renders, preventing downstream useEffect or child component rerenders that depend on identity equality.
3. **Avoiding recreation of functions** – you can also memoize callbacks with useCallback (a thin wrapper around useMemo).

Example

```
import { useMemo, useState } from "react";

function App() {
  const [count, setCount] = useState(0);

  // `total` is calculated once because the dependency array is empty.
  const total = useMemo(() => {
    console.log("Calculating...");
    let sum = 0;
    for (let i = 1; i <= 1000000000; i++) {
      sum += i;
    }
    return sum;
  }, []);

  return (
    <>
      <h2>{count}</h2>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <h1>{total}</h1>
    </>
  );
}
```

```
</>  
);  
}
```

What happens without useMemo?

- Each time count changes, App rerenders.
- The loop runs again, blocking the UI and printing *"Calculating..."* repeatedly.

With useMemo

- The loop runs **only once** (on mount). Subsequent renders reuse the cached sum, making the counter instant.

When not to use useMemo

- The computation is cheap – the overhead of storing a memoized value outweighs the benefit.
- You need the latest value on every render (e.g., reading mutable state inside the calculation).
- Relying on useMemo for sideeffects is an antipattern; sideeffects belong in useEffect.

React.memo

React.memo is a **higherorder component (HOC)** that wraps a **function component** and performs a **shallow comparison** of its props. If props have not changed, React skips rendering the component tree.

```
const MemoizedComponent = React.memo(function MyComponent(props) {  
  // render logic  
});
```

You can also supply a **custom comparison function** as the second argument:

```
export default React.memo(MyComponent, (prev, next) => {  
  // return true if props are equal → skip render  
});
```

Example

App.jsx

```
import React, { useState } from "react";
import Sum from "./Sum";

function App() {
  const [count, setCount] = useState(0);
  console.log("App Render");
  return (
    <>
      <h1>Counter: {count}</h1>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <Sum />
    </>
  );
}
export default App;
```

Sum.jsx

```
import React from "react";

// Wrap the component with React.memo so it only rerenders when its *props* change.
const Sum = React.memo(() => {
  function totalSum() {
    console.log("Sum rendering");
    let sum = 0;
    for (let i = 0; i <= 1000; i++) {
      sum += i;
    }
    return sum;
  }
  const totalSumValue = totalSum();
  return (
    <>
      <h2>Value : {totalSumValue}</h2>
    </>
  );
});

export default Sum;
```

What you observe

- Clicking *Increment* updates count → App rerenders (logs "App Render").
- **Without React.memo**, Sum would also rerender on every count change, printing "Sum rendering" each time.
- With React.memo, Sum **does not rerender** because it receives **no props** that change. The console shows "Sum rendering" only once (on initial mount).

When to use React.memo

1. **Pure presentational components** that receive props and have no internal state.
2. Components that perform **expensive calculations** or render large subtrees.
3. When the parent component updates frequently but the child's props stay the same.

Caveats

- The default shallow comparison only checks **reference equality** for objects/arrays. If you pass a newlycreated object each render, React.memo will think props changed. Use useMemo to memoize those objects.
- Overusing React.memo can add unnecessary memory overhead and make debugging harder.
- It does **not** help with **context** changes – if a component reads useContext, it will rerender when the context value changes regardless of memoization.

How to Combine useMemo & React.memo

Often you'll memoize **props** that you pass to a memoized child component:

```
function Parent() {  
  const [count, setCount] = useState(0);  
  
  // Memoize an object prop so its reference does not change on each render.  
  const config = useMemo(() => ({ step: 1, limit: 1000 }), []);  
}
```

```
return (  
  <>  
    <button onClick={() => setCount(c => c + 1)}>Inc</button>  
    <HeavyChild config={config} />  
  </>  
);  
}
```

```
const HeavyChild = React.memo(({ config }) => {  
  // heavy calculation that depends on config  
});
```

Here HeavyChild rerenders only when config actually changes.

Checklist for Effective Memoization

- ☐ Is the computation **expensive** enough to merit memoization?
- ☐ Is the function **pure** (no sideeffects, deterministic output)?
- ☐ Have you added **correct dependencies** to useMemo?
- ☐ Are you memoizing **props** (objects/arrays) before passing them to a React.memo component?
- ☐ Have you measured the performance impact (Chrome Profiler, React DevTools "Highlight updates")?
- ☐ Remember: **Never** rely on useMemo for data fetching; use useEffect or libraries like React Query.

useEffect

useEffect vs. useMemo

Hook	Primary Purpose	Returns	Runs When
useEffect	Perform sideeffects (data fetching, subscriptions, timers, manual DOM mutations).	Nothing (optional cleanup function).	After render – on mount, update, or unmount depending on dependency array.
useMemo	Memoize expensive computed values to avoid recalculation.	Memoized value.	During render – before the component's output is committed, only when dependencies change.

When to use useEffect

1. **Fetching data** from an API.
2. **Subscribing** to a WebSocket, event listeners, or external libraries.
3. **Setting up timers** (setTimeout, setInterval).
4. **Manually interacting with the DOM** (e.g., focusing an input, measuring layout).
5. **Cleaning up** resources (return a function that runs on unmount or before the next effect).

Basic Syntax

```
import { useEffect } from "react";
```

```
function Component({ userId }) {
  useEffect(() => {
    // sideeffect code runs after every render by default
    console.log("Component rendered");
    return () => {
      // optional cleanup runs before the next effect or on unmount
      console.log("Component will unmount or deps change");
    };
  }, []); // <- dependency array (explained later)

  return <div>...</div>;
}
```

Dependency Array – The Heart of Both Hooks

The **dependency array** tells React **what pieces of state or props** the hook depends on.

- **Empty array []** → run **once** (mount) and optionally clean up on unmount.
- **Omitted array** → run **after every render** (behaves like `componentDidUpdate` plus `componentDidMount`).
- **Specific dependencies** → run **only when those values change** (shallow compare using `===`).

Situations & Examples

1 Run once on mount – data fetch

```
useEffect(() => {
  fetch(`/api/user/${userId}`)
    .then(r => r.json())
    .then(setData);
}, []); // fetch only once
```

2 Rerun when a prop changes – fetch new data

```
useEffect(() => {
  fetch(`/api/items?category=${category}`)
    .then(r => r.json())
    .then(setItems);
}, [category]); // refetch when `category` changes
```

3 Set up a timer that restarts on a variable change

```
useEffect(() => {
  const id = setInterval(() => {
```

```
setSeconds(s => s + 1);
}, interval);
return () => clearInterval(id); // cleanup old interval
}, [interval]); // interval value is a dependency
```

4 Sync a computed value with a prop using useMemo

```
const expensive = useMemo(() => computeHeavy(data), [data]);
```

Here data is the dependency; expensive updates only when data changes.

useMemo – Recap of When It Runs

```
const memoized = useMemo(() => heavyComputation(a, b), [a, b]);
```

- **Runs during render** – before the component output is committed.
- **Returns a cached value**; React **does not** invoke the function again unless one of the dependencies has changed.
- **Never for sideeffects** – the function must be pure.

Direct Comparison – useEffect vs. useMemo

Aspect	useEffect	useMemo
What it does	Executes sideeffects after the render cycle (e.g., API calls, subscriptions).	Computes and caches a value during render.
Return value	Optional cleanup function ; otherwise undefined.	Memoized value (any type).
When it runs	After the DOM is painted. Runs on mount, on updates (if deps change),	Synchronously during render. Runs only when dependencies change.

	and on unmount (cleanup).	
Typical usecases	Data fetching, timers, DOM manipulation, event listeners.	Expensive calculations, stable object/array references, derived data for useEffect or child props.
Dependency array	Controls when the sideeffect runs. Empty → once. Omitted → every render.	Controls when the value is recomputed. Same semantics as useEffect.
Can cause UI block?	No (runs after paint), but heavy work inside may still block if not async.	Yes – because it runs <i>during</i> render; heavy work can freeze UI if not memoized correctly.
Cleanup	Yes – return a function to release resources.	No – not applicable.
Can be skipped	Skipped if deps didn't change.	Skipped if deps didn't change.

RealWorld Example Combining Both

```
import { useState, useEffect, useMemo } from "react";

function Search({ query }) {
  // Memoize the URL – avoids creating a new string on every render.
  const url = useMemo(() => `/api/search?q=${encodeURIComponent(query)}`, [query]);

  const [results, setResults] = useState([]);
  const [loading, setLoading] = useState(false);
```

```
// Effect performs the fetch sideeffect.
useEffect(() => {
  if (!query) return; // guard
  setLoading(true);
  fetch(url)
    .then(r => r.json())
    .then(data => setResults(data))
    .finally(() => setLoading(false));
}, [url]); // rerun only when the memoized URL changes

return (
  <div>
    {loading ? <p>Loading...</p> : results.map(r => <p key={r.id}>{r.title}</p>)}
  </div>
);
}
```

- useMemo guarantees the **URL reference** stays stable unless query changes, preventing unnecessary effect runs.
- useEffect handles the **asynchronous fetch** and cleanup (if we added an abort controller).

DependencyArray BestPractices Checklist

- **List every value** used inside the hook that comes from props or state.
- **Do NOT ignore** functions defined in the component – they are new on each render, so wrap them with useCallback or move them outside.
- **Never put a nonstable value** (e.g., an object literal) directly in the array; memoize it with useMemo first.
- **For useEffect**, if you only need to run once, use [] and **ensure** any variables referenced are either constant or intentionally stale.
- **For useMemo**, ensure the computation is **pure**; sideeffects inside will be ignored by React and can cause bugs.

Routing

What is Routing?

Routing is the process of determining what content to show a user based on the URL they request. Every time you type a URL, click a link, or submit a form, some system has to decide: *"Given this URL, what should the user see?"*

There are two fundamentally different ways this can happen: **server-side routing** and **client-side routing**.

Server-Side Routing

Server-side routing is the traditional, original model of the web.

How it works:

1. The browser sends an HTTP request to the server for a specific URL (e.g. /about).
2. The server looks at the URL, decides which page/template to build, and generates a full HTML document.
3. The server sends the complete HTML page back to the browser.
4. The browser throws away the old page and loads the new one from scratch (a full page reload).
5. This repeats for every single link the user clicks.

Example flow:

None

1. User clicks "About" link
2. ↓
3. Browser sends GET /about to server
4. ↓
5. Server builds a new HTML page for /about
6. ↓
7. Browser reloads completely with new HTML

Characteristics of server-side routing:

- Every navigation triggers a **full page reload**.
- The server does all the work of deciding what HTML to send.
- Good for SEO by default, since search engine crawlers receive fully-formed HTML for every URL.

- Initial page load can be fast because the browser receives ready-to-render HTML.
 - Navigating between pages feels slower because the whole page (CSS, JS, images, layout) reloads every time.
 - Used by traditional multi-page applications (MPAs) — e.g., classic PHP, Django, or Ruby on Rails websites.
-

Client-Side Routing

Client-side routing is the model used by modern Single Page Applications (SPAs), including most React apps.

How it works:

1. The browser loads **one single HTML page** from the server (usually just once).
2. Along with that page, a JavaScript bundle (your React app) is downloaded.
3. From then on, when the user clicks a link, JavaScript running in the browser intercepts the click.
4. Instead of asking the server for a new page, JavaScript:
 - Updates the URL in the address bar (using the browser's History API).
 - Swaps out the components rendered on the screen.
5. No full page reload happens — only the necessary parts of the UI update.

Example flow:

None

8. User clicks "About" link
9. ↓
10. JavaScript (React Router) intercepts the click
11. ↓
12. URL is updated to /about (no request to server)
13. ↓
14. React swaps in the <About /> component instantly

Characteristics of client-side routing:

- Navigation is **instant** — no full page reload, so it feels app-like and fast.
- The browser only ever loads one HTML shell; the rest is rendered dynamically by JavaScript.
- State (like a shopping cart, a logged-in user, a form being filled out) can be preserved across "page" changes, since the page never actually reloads.

- SEO can be harder, because search engines may see an almost-empty HTML shell unless extra steps (like server-side rendering) are taken.
 - Requires a library like **React Router**, **Vue Router**, or **Angular Router** to manage the illusion of multiple pages within one HTML document.
 - The server usually still needs to be configured to always return the same `index.html` for any route (a "fallback"), so that refreshing a client-side route like `/dashboard/profile` doesn't cause a server 404.
-

Client-Side Routing vs Server-Side Routing

Aspect	Server-Side Routing	Client-Side Routing
Who decides what to render	The server	The browser (JavaScript)
Page reload on navigation	Full reload every time	No reload — instant swap
Speed of navigation	Slower (new request each time)	Fast (no network round-trip needed)
Initial load	Fast, ready-made HTML	Slower — must download JS bundle first
SEO	Naturally strong	Weaker unless SSR/SSG is used
App type	Multi-Page Application (MPA)	Single Page Application (SPA)

State persistence across pages	Lost on reload	Preserved (app never reloads)
Example technologies	PHP, Django, Rails, plain HTML sites	React Router, Vue Router, Angular Router
URL handling	Server generates HTML per URL	JavaScript updates URL + swaps components

Key idea to remember:

Server-side routing = *"Ask the server for a new page."* Client-side routing = *"Let JavaScript in the browser fake a new page."*

Many modern frameworks (like Next.js) actually blend both — they can render pages on the server first (for speed and SEO) and then take over with client-side routing once the JavaScript loads. This hybrid approach is often called **server-side rendering (SSR)** combined with **client-side navigation**.

React Router — Core Concepts

React Router is the standard library used to implement **client-side routing** in React applications, allowing multiple pages/views without reloading the browser.

Install it:

None

```
15. npm install react-router-dom
```

BrowserRouter

BrowserRouter is the main wrapper that enables routing in your React application.

None

```
16. import { BrowserRouter } from "react-router-dom";
17.
18. function App() {
19.   return (
20.     <BrowserRouter>
21.       <AppRoutes />
22.     </BrowserRouter>
23.   );
24. }
```

Think of it as: BrowserRouter = enables React Router.

Usually, we wrap the entire application with it.

Routes

Routes is a container that holds all your routes.

None

```
25. <Routes>
26.   <Route path="/" element={<Home />} />
27.   <Route path="/about" element={<About />} />
28. </Routes>
```

Think: Routes = collection of routes.

Route

Route defines which component should appear for a particular URL.

None

```
29. <Route path="/about" element={<About />} />
```

Here:

- `path="/about"` → URL
- `element={<About />}` → component to display

Think: Route = URL → Component.

Link

Link is used for navigation without refreshing the page.

None

```
30. import { Link } from "react-router-dom";  
31.  
32. <Link to="/about">About</Link>
```

Instead of:

None

```
33. <a href="/about">About</a>
```

Use:

None

```
34. <Link to="/about">About</Link>
```

For React Router applications, Link is generally preferred because it performs client-side navigation.

NavLink

NavLink is similar to Link, but it is especially useful for navigation menus because it can tell you when the link is active.

None

```
35. <NavLink to="/home">Home</NavLink>
```

You can apply styling to the active link:

None

```
36. <NavLink
37.   to="/home"
38.   className={({ isActive }) =>
39.     isActive ? "active" : ""
40.   }
41. >
42.   Home
43. </NavLink>
```

Think:

- Link → navigation
 - NavLink → navigation + active state
-

useNavigate()

useNavigate() is used when you want to navigate programmatically using JavaScript.

None

```
44. import { useNavigate } from "react-router-dom";
45.
46. function Login() {
47.   const navigate = useNavigate();
48.
49.   function handleLogin() {
50.     navigate("/dashboard");
51.   }
52.
53.   return <button onClick={handleLogin}>Login</button>;
54. }
```

Here, clicking the button takes the user to /dashboard.

Think:

- Link → user clicks a link
- `useNavigate` → JavaScript decides when to navigate

You can also go backward:

None

```
55. navigate(-1);
```

Forward:

None

```
56. navigate(1);
```

`useParams()`

Used to get dynamic values from the URL.

For example:

None

```
57. <Route path="/student/:id" element={<Student />} />
```

URL: `/student/101`

Component:

None

```
58. import { useParams } from "react-router-dom";  
59.  
60. function Student() {  
61.   const { id } = useParams();  
62.  
63.   return <h1>Student ID: {id}</h1>;  
64. }
```

Output: Student ID: 101

Think: `useParams()` = get values from URL parameters.

Dynamic Routes

A route can contain a dynamic parameter:

None

```
65. <Route path="/product/:id" element={<Product />} />
```

These URLs can all match:

None

```
66. /product/10
```

```
67. /product/20
```

```
68. /product/50
```

`id` changes dynamically.

Nested Routes

Routes can exist inside other routes.

Example:

None

```
69. <Routes>
```

```
70.   <Route path="/dashboard" element={<Dashboard />}>
```

```
71.     <Route path="profile" element={<Profile />} />
```

```
72.     <Route path="settings" element={<Settings />} />
```

```
73.   </Route>
```

```
74. </Routes>
```

URLs:

None

```
75. /dashboard/profile  
76. /dashboard/settings
```

The parent component generally renders an `<Outlet />`:

None

```
77. function Dashboard() {  
78.   return (  
79.     <>  
80.       <h1>Dashboard</h1>  
81.       <Outlet />  
82.     </>  
83.   );  
84. }
```

Outlet

Outlet tells React Router: *"Render the child route here."*

Example:

None

```
85. function Dashboard() {  
86.   return (  
87.     <div>  
88.       <h1>Dashboard</h1>  
89.       <Outlet />  
90.     </div>  
91.   );  
92. }
```

If the URL is `/dashboard/profile`, then `<Profile />` will appear where `<Outlet />` is located.

Navigate

Navigate is a component used for navigation/redirects.

None

```
93. import { Navigate } from "react-router-dom";
94.
95. function ProtectedRoute() {
96.   const isLoggedIn = false;
97.
98.   if (!isLoggedIn) {
99.     return <Navigate to="/login" />;
100.   }
101.
102.   return <Dashboard />;
103. }
```

Think:

- `useNavigate()` → function
 - `<Navigate />` → component
-

useLocation()

`useLocation()` gives information about the current URL/location.

None

```
104. import { useLocation } from "react-router-dom";
105.
106. function App() {
107.   const location = useLocation();
108.
109.   console.log(location);
110.
111.   return <h1>Current Page</h1>;
112. }
```

It provides information such as:

- pathname
- search
- hash
- state

For example, for /products?category=mobile:

- pathname: /products
- search: ?category=mobile

useSearchParams()

Used to work with query parameters.

URL: /products?category=mobile

Code:

None

```
113. const [searchParams] = useSearchParams();
114.
115. const category = searchParams.get("category");
116.
117. console.log(category);
```

Output: mobile

useRoutes()

Instead of writing <Routes> and <Route> directly, you can define routes as JavaScript objects.

None

```
118. const routes = [
119.   {
120.     path: "/",
```

```

121.     element: <Home />
122.   },
123.   {
124.     path: "/about",
125.     element: <About />
126.   }
127. ];

```

Then:

None

```

128.   const element = useRoutes(routes);

```

This is useful for larger applications.

Outlet + Nested Routing (Common Structure)

A very common structure is:

None

```

129. App
130. |
131. └─ Dashboard
132.     |
133.     └─ Profile
134.         |
135.         └─ Settings

```

Routes:

None

```

136. <Routes>
137.   <Route path="/dashboard" element={<Dashboard />}>
138.     <Route path="profile" element={<Profile />} />

```

```
139.     <Route path="settings" element={<Settings />} />
140.   </Route>
141. </Routes>
```

Dashboard:

None

```
142. function Dashboard() {
143.   return (
144.     <>
145.       <h1>Dashboard</h1>
146.       <Outlet />
147.     </>
148.   );
149. }
```

index Route

An index route is the default child route.

None

```
150. <Route path="/dashboard" element={<Dashboard />}>
151.   <Route index element={<Home />} />
152.   <Route path="profile" element={<Profile />} />
153. </Route>
```

When the user visits /dashboard, Home is displayed.

When they visit /dashboard/profile, Profile is displayed.

Catch-All / 404 Route (*)

You can create a route for pages that don't exist:

None

```
154. <Route path="*" element={<NotFound />} />
```

For example:

None

```
155. /abc
156. /random
157. /hello
```

If no other route matches, NotFound appears.

Note: This catch-all only works because of client-side routing. In a server-side routed app, an unmatched URL would return a real HTTP 404 from the server itself.

Complete Basic Example

A beginner-friendly React Router application could look like this:

None

```
158. import {
159.   BrowserRouter,
160.   Routes,
161.   Route,
162.   Link
163. } from "react-router-dom";
164.
165. function Home() {
166.   return <h1>Home</h1>;
167. }
168.
169. function About() {
170.   return <h1>About</h1>;
171. }
172.
```

```
173. function Contact() {
174.     return <h1>Contact</h1>;
175. }
176.
177. function App() {
178.     return (
179.         <BrowserRouter>
180.
181.         <nav>
182.             <Link to="/">Home</Link>
183.             <Link to="/about">About</Link>
184.             <Link to="/contact">Contact</Link>
185.         </nav>
186.
187.         <Routes>
188.             <Route path="/" element={<Home />} />
189.             <Route path="/about" element={<About />} />
190.             <Route path="/contact" element={<Contact />} />
191.         </Routes>
192.
193.     </BrowserRouter>
194. );
195. }
196.
197. export default App;
```

Quick Reference Table

Term	Purpose
Server-side routing	Server generates and sends a new HTML page for every URL (full reload)

Client-side routing	JavaScript in the browser swaps components and updates the URL (no reload)
BrowserRouter	Enables routing
Routes	Holds routes
Route	Maps URL to component
Link	Navigate without reload
NavLink	Link + active styling
useNavigate()	Navigate using JavaScript
useParams()	Get dynamic URL values
useLocation()	Get current URL information
useSearchParams()	Work with query parameters
Navigate	Redirect
Outlet	Display nested route

index

Default nested route

*

Catch-all / 404 route

useRoutes()

Define routes as objects

REST

Fullstack Topics:

1. React Router, useParams, useNavigate, useSearchParams
2. Context API, Providers, useContext
3. REST APIs
4. RBAC, Error handling(Status codes)

REST APIs, HTTP Methods, and Axios

What is an API?

An **API (Application Programming Interface)** is a set of rules that lets one piece of software talk to another. On the web, this usually means your frontend (React app) talking to a backend server to get or send data — for example, fetching a list of products, or submitting a login form.

What is a REST API?

REST (Representational State Transfer) is a set of design principles for building APIs. A REST API exposes **resources** (like users, products, orders) as URLs, and lets clients interact with those resources using standard HTTP methods.

Key ideas behind REST:

- **Resources are identified by URLs.** Example: `/users`, `/users/5`, `/products/12/reviews`.
- **Stateless.** Each request from the client contains all the information the server needs — the server doesn't remember previous requests.
- **Standard HTTP methods represent actions.** Instead of inventing new verbs, REST reuses HTTP methods (GET, POST, PUT, PATCH, DELETE) to mean create/read/update/delete.
- **Data is usually exchanged as JSON.** Most modern REST APIs send and receive data in JSON format.
- **Predictable, consistent URL patterns.** A well-designed REST API follows a pattern like:

None

```
GET    /users    → get all users
GET    /users/5  → get user with id 5
POST   /users    → create a new user
PUT    /users/5  → replace user 5 entirely
PATCH /users/5  → update part of user 5
DELETE /users/5  → delete user 5
```

Think: REST = a convention for organizing URLs + HTTP methods so that any client (web, mobile, another server) can predictably work with your data.

HTTP Methods (The Verbs of the Web)

Every HTTP request has a **method** that tells the server what kind of action the client wants to perform.

Method	Purpose	Typical Use Case	Sends a Body?
GET	Retrieve data	Fetch a list of products, get a user's profile	No
POST	Create new data	Submit a signup form, create a new blog post	Yes
PUT	Replace an existing resource entirely	Overwrite a user's full profile with new data	Yes

PATCH	Partially update an existing resource	Update just a user's email address	Yes
DELETE	Remove a resource	Delete a comment, remove an item from a cart	Usually no

A simple way to remember (CRUD mapping):

None

Create → POST

Read → GET

Update → PUT / PATCH

Delete → DELETE

- **GET** requests should never change data on the server — they're considered "safe" and are often cached by browsers.
- **POST**, **PUT**, **PATCH**, and **DELETE** typically change something on the server, so they are not cached the same way.
- **PUT vs PATCH**: PUT expects the *entire* resource to be sent (it replaces it), while PATCH only sends the *fields that changed*.

HTTP Status Codes (what the server replies with):

Range	Meaning	Common Examples
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 304 Not Modified
4xx	Client error (you made a mistake)	400 Bad Request, 401 Unauthorized, 404 Not Found
5xx	Server error (the server made a mistake)	500 Internal Server Error, 503 Service Unavailable

Axios

Axios is a popular JavaScript library used to make HTTP requests from the browser (or Node.js) to a server. It's one of the most common ways React apps talk to REST APIs.

Install it:

None

```
npm install axios
```

Why use Axios instead of the built-in `fetch()`?

Feature	<code>fetch()</code>	Axios
Built into the browser	Yes	No (external library)
Automatic JSON parsing	No — must call <code>.json()</code> manually	Yes — response data is parsed automatically
Request/response interceptors	No (must build manually)	Yes, built in
Automatic request cancellation	Limited	Built-in support (<code>AbortController</code> / cancel tokens)
Handles errors for bad status codes (4xx/5xx)	No — you must check <code>response.ok</code> yourself	Yes — throws an error automatically
Timeout support	Manual	Built-in timeout option
Works in Node.js	Needs polyfill	Yes, natively

Making Requests with Axios

Basic GET request:

jsx

None

```
import axios from "axios";

function getUsers() {
  axios.get("https://api.example.com/users")
    .then((response) => {
      console.log(response.data);
    });
}
```

```

    })
    .catch((error) => {
      console.error(error);
    });
  });
}

```

GET request using async/await (preferred in modern React):

jsx

None

```

import axios from "axios";
import { useEffect, useState } from "react";

function Users() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    async function fetchUsers() {
      try {
        const response = await
axios.get("https://api.example.com/users");
        setUsers(response.data);
      } catch (error) {
        console.error("Error fetching users:", error);
      }
    }

    fetchUsers();
  }, []);

  return (
    <ul>
      {users.map((user) => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}

```

```
);  
}
```

POST request (creating data):

jsx

```
None  
async function createUser(newUser) {  
  const response = await  
    axios.post("https://api.example.com/users", newUser);  
  console.log(response.data);  
}  
  
createUser({ name: "Ayaan", email: "ayaan@example.com" });
```

PUT request (replacing data):

jsx

```
None  
async function replaceUser(id, updatedUser) {  
  const response = await  
    axios.put(`https://api.example.com/users/${id}`, updatedUser);  
  console.log(response.data);  
}
```

PATCH request (partially updating data):

jsx

```
None  
async function updateUserEmail(id, newEmail) {  
  const response = await  
    axios.patch(`https://api.example.com/users/${id}`, {  
      email: newEmail  
    });  
  console.log(response.data);  
}
```

DELETE request:

jsx

None

```
async function deleteUser(id) {  
  await axios.delete(`https://api.example.com/users/${id}`);  
  console.log("User deleted");  
}
```

Setting Headers and Config with Axios

Sometimes you need to send extra information with a request, like an authentication token:

jsx

None

```
axios.get("https://api.example.com/profile", {  
  headers: {  
    Authorization: `Bearer ${token}`  
  }  
});
```

Creating an Axios Instance

For larger apps, it's common to create a pre-configured Axios instance so you don't repeat the base URL and headers everywhere:

jsx

None

```
import axios from "axios";  
  
const api = axios.create({  
  baseURL: "https://api.example.com",  
  timeout: 5000,  
  headers: {  
    "Content-Type": "application/json"  
  }  
})
```

```
});  
  
export default api;
```

Then anywhere in the app:

jsx

```
None  
import api from "./api";  
  
const response = await api.get("/users");
```

Axios Quick Reference

Method	Axios Syntax	Purpose
GET	<code>axios.get(url)</code>	Retrieve data
POST	<code>axios.post(url, data)</code>	Create new data
PUT	<code>axios.put(url, data)</code>	Replace existing data
PATCH	<code>axios.patch(url, data)</code>	Partially update data
DELETE	<code>axios.delete(url)</code>	Remove data
Response data	<code>response.data</code>	The actual payload returned by the server
Response status	<code>response.status</code>	HTTP status code (e.g. 200, 404)
Custom config	<code>axios.get(url, config)</code>	Pass headers, params, timeout, etc.
Reusable instance	<code>axios.create({...})</code>	Pre-configure baseURL, headers, timeout

Practice_Questions

Question 1: Online Food Ordering Application

Scenario

You have joined a startup that is developing a simple Online Food Ordering Application. When a customer logs in, their basic information is available at the top-level App component. Different components of the application need access to this information.

Initially, the development team passed customer information through multiple components using props. As the application grew, this resulted in prop drilling, because some intermediate components were receiving customer data only to pass it on to the next component.

Your task is to first demonstrate this problem, and then improve the application using the Context API and a custom hook. You will also use `useEffect` to perform an action whenever the customer's cart changes.

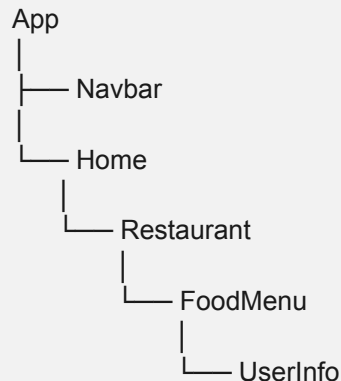
Given Data

Assume the logged-in customer information is:

```
const user = {  
  name: "Rahul Sharma",  
  email: "rahul@gmail.com",  
  location: "Chandigarh"  
};
```

Component Structure

The application should have approximately the following component structure:



Requirements

1. Demonstrate Prop Drilling

- Store the user object inside the App component.
- Pass it from App → Home → Restaurant → FoodMenu → UserInfo using props only.
- Home, Restaurant, and FoodMenu do not need the user information themselves; they only forward it to the next component.
- Inside UserInfo, display the customer's name, email, and location.

2. Solve the Problem Using the Context API

- Create a UserContext using the Context API and provide the logged-in user's information from the App component.
- Modify the application so that UserInfo can directly access the customer information without receiving it as a prop through Home, Restaurant, or FoodMenu.
- Do this WITHOUT deleting your prop-drilling solution from Requirement 1 — keep both versions in the project so the improvement is visible (see Submission Guidelines for how to structure this).

3. Create a Custom Hook

- Create a custom hook called useUser().
- The hook should access UserContext and return the logged-in customer's information.
- Use useUser() inside the UserInfo component instead of calling useContext() directly there.

4. Implement a Shopping Cart

- Inside FoodMenu, display at least three food items, for example:

Pizza	₹299	[Add to Cart]
Burger	₹149	[Add to Cart]
Pasta	₹199	[Add to Cart]

- When the customer clicks Add to Cart, update a cartCount state variable using useState.
- Display the current number of items in the Navbar, for example:

Welcome, Rahul Sharma

Cart: 2 items

5. Use useEffect

- Whenever the number of items in the cart changes, use useEffect to print a message to the browser console, for example:

Cart updated. Total items: 1

- After another item is added:

Cart updated. Total items: 2

- Make sure this effect runs only when cartCount changes.

Expected Behaviour

When the application starts, the user should see something similar to:

QUICK BITE - FOOD ORDERING APP

Welcome, Rahul Sharma
Location: Chandigarh Cart: 0

FOOD MENU

Pizza	₹299 [Add to Cart]
Burger	₹149 [Add to Cart]
Pasta	₹199 [Add to Cart]

Customer Details

Name: Rahul Sharma
Email: rahul@gmail.com
Location: Chandigarh

After clicking Add to Cart twice, the Navbar should update to:

Cart: 2 items

and the browser console should show:

Cart updated. Total items: 2

Question 2: Movie Ticket Booking Application

Scenario

You are working on a Movie Ticket Booking Application for a startup called CineGo. When a customer logs in, their basic information is available at the top-level App component. Several components deeper in the tree need this information to personalise the booking experience.

The first version of the app was built by passing customer information through multiple intermediate components using props, even though most of those components never actually use the data themselves. This is the same prop drilling problem the team ran into with the food ordering app.

As before, your task is to first demonstrate the problem, then fix it using the Context API and a custom hook, and finally use `useEffect` to react whenever the customer's seat selection changes.

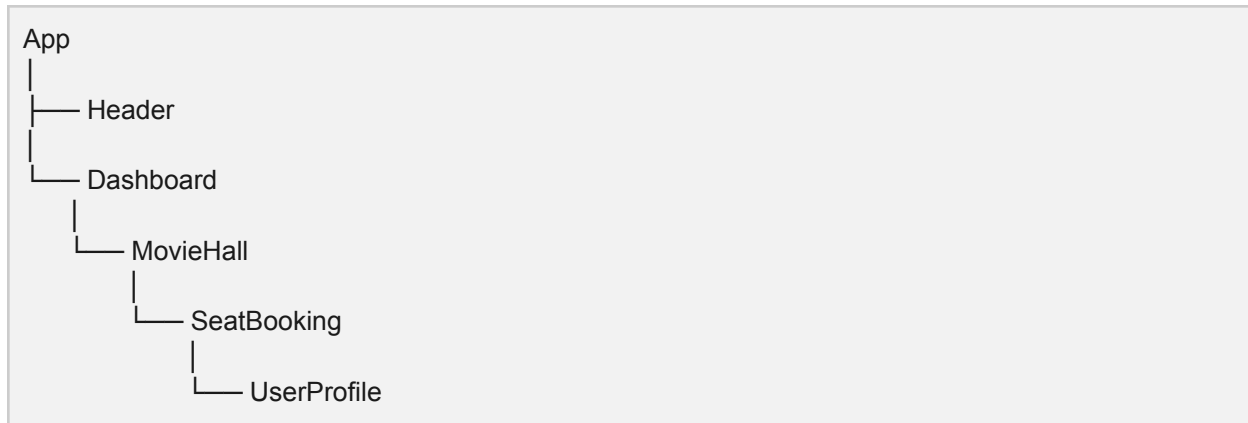
Given Data

Assume the logged-in customer information is:

```
const user = {  
  name: "Ananya Verma",  
  email: "ananya@gmail.com",  
  city: "Mumbai"  
};
```

Component Structure

The application should have approximately the following component structure:



Requirements

1. Demonstrate Prop Drilling

- Store the user object inside the App component.
- Pass it from App → Dashboard → MovieHall → SeatBooking → UserProfile using props only.
- Dashboard, MovieHall, and SeatBooking do not need the user information themselves; they only forward it to the next component.
- Inside UserProfile, display the customer's name, email, and city.

2. Solve the Problem Using the Context API

- Create a UserContext using the Context API and provide the logged-in user's information from the App component.
- Modify the application so that UserProfile can directly access the customer information without receiving it as a prop through Dashboard, MovieHall, or SeatBooking.
- Do this WITHOUT deleting your prop-drilling solution from Requirement 1 — keep both versions in the project so the improvement is visible (see Submission Guidelines for how to structure this).

3. Create a Custom Hook

- Create a custom hook called useUser().
- The hook should access UserContext and return the logged-in customer's information.

- Use `useUser()` inside the `UserProfile` component instead of calling `useContext()` directly there.

4. Implement Seat Selection

- Inside `SeatBooking`, display at least three seats, for example:

```
Seat A1    ₹250    [Select Seat]
Seat A2    ₹250    [Select Seat]
Seat B1    ₹350    [Select Seat]
```

- When the customer clicks `Select Seat`, update a `selectedSeats` state variable using `useState`.
- Display the current number of selected seats in the Header, for example:

```
Welcome, Ananya Verma

Seats Selected: 2
```

5. Use `useEffect`

- Whenever the number of selected seats changes, use `useEffect` to print a message to the browser console, for example:

```
Booking updated. Seats selected: 1
```

- After another seat is selected:

```
Booking updated. Seats selected: 2
```

- Make sure this effect runs only when `selectedSeats` changes.

Expected Behaviour

When the application starts, the user should see something similar to:

CINEGO - MOVIE TICKET BOOKING

Welcome, Ananya Verma

City: Mumbai

Seats Selected: 0

SELECT YOUR SEATS

Seat A1 ₹250
[Select Seat]

Seat A2 ₹250
[Select Seat]

Seat B1 ₹350
[Select Seat]

Customer Details

Name: Ananya Verma

Email: ananya@gmail.com

City: Mumbai

After selecting two seats, the Header should update to:

Seats Selected: 2

and the browser console should show:

Booking updated. Seats selected: 2

Question 3: Online Learning Platform

Scenario

You are building the student dashboard for an EdTech startup called SkillNest. When a student logs in, their basic information is available at the top-level App component. Several components deeper in the tree need this information to personalise the dashboard.

The first version of the app was built by passing the student's information through multiple intermediate components using props, even though most of those components never actually use the data themselves. This is the same prop drilling problem the team ran into in the earlier apps.

As before, your task is to first demonstrate the problem, then fix it using the Context API and a custom hook, and finally use `useEffect` to react whenever the number of enrolled courses changes.

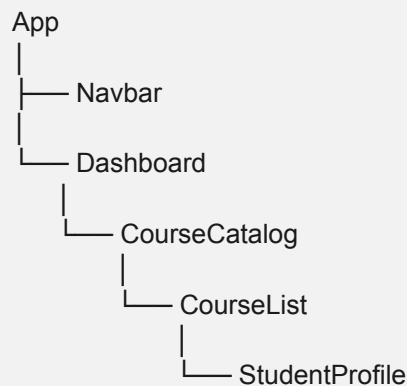
Given Data

Assume the logged-in student information is:

```
const user = {  
  name: "Karan Mehta",  
  email: "karan@gmail.com",  
  program: "Web Development"  
};
```

Component Structure

The application should have approximately the following component structure:



Requirements

1. Demonstrate Prop Drilling

- Store the user object inside the App component.
- Pass it from App → Dashboard → CourseCatalog → CourseList → StudentProfile using props only.
- Dashboard, CourseCatalog, and CourseList do not need the user information themselves; they only forward it to the next component.

- Inside StudentProfile, display the student's name, email, and program.

2. Solve the Problem Using the Context API

- Create a useContext using the Context API and provide the logged-in student's information from the App component.
- Modify the application so that StudentProfile can directly access the student information without receiving it as a prop through Dashboard, CourseCatalog, or CourseList.
- Do this WITHOUT deleting your prop-drilling solution from Requirement 1 — keep both versions in the project so the improvement is visible (see Submission Guidelines for how to structure this).

3. Create a Custom Hook

- Create a custom hook called useUser().
- The hook should access useContext and return the logged-in student's information.
- Use useUser() inside the StudentProfile component instead of calling useContext() directly there.

4. Implement Course Enrollment

- Inside CourseList, display at least three courses, for example:

React Basics	₹999	[Enroll]
Node.js Essentials	₹1199	[Enroll]
UI/UX Design	₹799	[Enroll]

- When the student clicks Enroll, update an enrolledCount state variable using useState.
- Display the current number of enrolled courses in the Navbar, for example:

Welcome, Karan Mehta

Enrolled: 2 courses

5. Use useEffect

- Whenever the number of enrolled courses changes, use useEffect to print a message to the browser console, for example:

Enrollment updated. Total courses: 1

- After enrolling in another course:

Enrollment updated. Total courses: 2

- Make sure this effect runs only when enrolledCount changes.

Expected Behaviour

When the application starts, the user should see something similar to:

SKILLNEST - ONLINE LEARNING PLATFORM

Welcome, Karan Mehta
Program: Web Development Enrolled: 0

COURSE CATALOG

React Basics ₹999
[Enroll]

Node.js Essentials ₹1199
[Enroll]

UI/UX Design ₹799
[Enroll]

Student Details

Name: Karan Mehta
Email: karan@gmail.com
Program: Web Development

After enrolling in two courses, the Navbar should update to:

Enrolled: 2 courses

and the browser console should show:

Enrollment updated. Total courses: 2

Question 4: Smart Campus Event Dashboard

Scenario

You are developing a Smart Campus Event Dashboard for a university. Students should be able to browse upcoming events, mark events as favourites, filter events by category, search for events, and view their own profile information alongside the event list.

This project builds on the component-communication concepts you have already practised (prop drilling, Context API, custom hooks) and adds three new ideas: fetching data from an API, managing related pieces of state together with useReducer, and avoiding unnecessary re-renders with React.memo.

Given Data

Assume the logged-in student's information is:

```
const student = {  
  name: "Riya",  
  department: "Computer Science",  
  semester: 6  
};
```

Component Structure

You may follow this structure, or adapt it as long as the same responsibilities are kept separate:

```
App  
|  
|-- StudentProvider  
|  
|-- Dashboard  
|   |-- Header  
|   |-- FilterButtons  
|   |-- SearchBar  
|   |-- EventList  
|       |-- EventCard (wrapped in React.memo)  
|   |  
|   |-- FavouriteCount  
|
```

```
|-- StudentInfo
```

Requirements

1. Fetch Events from an API

- Fetch data from a public API such as <https://jsonplaceholder.typicode.com/posts> (any similar list-returning API is acceptable).
- Transform each raw result into an event object with the shape: id, title, category, description, and isFavourite (initially false).
- Since the API has no real category, assign one yourself while transforming the data — for example, cycle through Technical, Cultural, Sports, and Workshop based on the item's id.
- Display the transformed events as cards inside EventList.

2. Manage Event State with useReducer.

- Replace what would otherwise be several separate useState calls (events, favourites, active filter, search term) with a single useReducer.
- Your reducer must support at least these action types:
- SET_EVENTS — loads the fetched events into state.
- ADD_FAVOURITE / REMOVE_FAVOURITE — toggles isFavourite for a given event id.
- FILTER_EVENTS — sets the active category filter.
- CLEAR_FILTER — resets the filter back to "All".
- Example dispatch call:

```
dispatch({  
  type: "ADD_FAVOURITE",  
  payload: eventId  
});
```

- The UI must re-render correctly in response to every action above.

3. Use the Context API for Student Information

- Create a StudentContext that stores the student object (name, department, semester).

- Provide it once, near the top of the tree (e.g. in a `StudentProvider` around `Dashboard` and `StudentInfo`).
- `StudentInfo` — nested deep inside the component tree in the example structure — must read the student's details directly from context, with no student-related prop passed down through any component in between.

4. Add Category Filtering

- Provide filter buttons: All | Technical | Cultural | Sports | Workshop.
- Selecting a category should display only matching events.
- The active category must live in your reducer's state (set via `FILTER_EVENTS` / `CLEAR_FILTER`), not in a separate `useState`.

5. Create a Favourite Counter

- At the top of the dashboard, display a header such as:

```
Welcome Riya
Favourite Events: 3
```

- The count must update immediately whenever a student favourites or unfavourites an event.

6. Optimise Event Cards with `React.memo`

- Wrap `EventCard` in `React.memo`, for example:

```
const EventCard = React.memo(function EventCard(props) {
  // ...
});
```

- `EventCard` should only receive the specific props it actually needs to render.
- Add an unrelated piece of UI state elsewhere in the app — for example a "Toggle Student Details" button that shows/hides `StudentInfo`.
- In a short written note (a few sentences, submitted alongside your code) explain why `EventCard` should NOT re-render when this unrelated state changes, given that its own props have not changed.

Bonus Challenge — Search Events

- Add a search box, for example:

Search: React

- Typing a search term should display only events whose title or description contains that term (case-insensitive).
- The search term must be stored and applied through your existing reducer — do not create a second, independent state-management mechanism for search.

Expected Behaviour

On load, after events have been fetched, the dashboard should look roughly like:

SMART CAMPUS EVENT DASHBOARD

Welcome Riya
Favourite Events: 0

[All] [Technical] [Cultural] [Sports] [Workshop]
Search: _____

Tech Talk: Intro to React
Category: Technical
A beginner-friendly session on React...
[Add to Favourites]

Annual Cultural Fest
Category: Cultural
Music, dance and drama competitions...
[Add to Favourites]

[Toggle Student Details]

Name: Riya
Department: Computer Science
Semester: 6

After clicking "Add to Favourites" on one event, the header should update to:

Favourite Events: 1

After selecting "Technical" from the filter buttons, only Technical events should remain visible.

After typing "React" into the search box, only events whose title or description mentions "React" should remain visible — combined with whatever filter is currently active.